

Izveštaj - Luka Petković SV16/2021

U ovom izveštaju analiziraćemo performanse simulacije dvostrukog klatna i upoređićemo skalabilnost rešenja u Python i Rust programskim jezicima. Kod Python implementacije koristio sam multiprocessing biblioteku za paralelizaciju, dok sam kod Rust-a koristio `std::threads`.

Eksperimenti su sprovedeni na laptopu sa sledećim konfiguracijama:

Procesor	AMD Ryzen 7 5700U
Broj jezgara	8 fizičkih, 16 logičkih
Radni takt	1.80 GHz
Cache memorija	L2 – 4MB, L3 - 8MB
RAM memorija	8GB
Operativni sistem	Windows 11

Treba uzeti u obzir da je hardver laptopa dosta zastareo, i da možda neće prikazati tako dobre performanse kao laptopovi novije generacije.

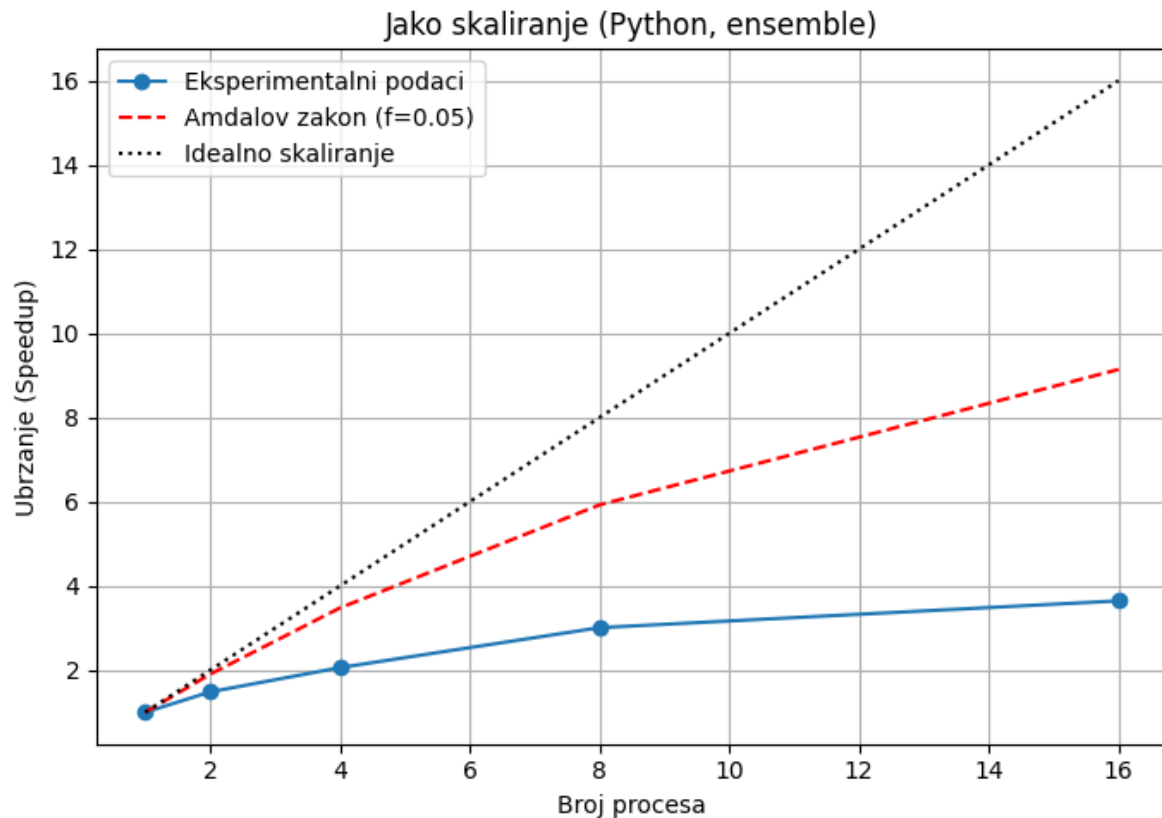
Analiza jakog skaliranja – Python

Eksperiment jakog skaliranja se vrši sa fiksnim ukupnim poslom. U slučaju dvostrukog klatna, treba da prikazemo sva stanja u toku 60 sekundi izvršavanja (**dt=0.001**, **steps=60000**). Stanja prikazujemo u csv fajlu gde imamo 6 osnovnih promenljivih: **t** – proteklo vreme, **teta1** – ugao prvog dela sa vertikalom, **teta2** – ugao drugog dela sa vertikalom, **omega1** – ugaona brzina prve ruke klatna, **omega2** – ugaona brzina druge ruke klatna, **energy** – ukupna energija sistema. Sve ove promenljive se kroz vreme menjaju, osim energy (ukupne energije), koja bi trebalo da ostane približno ista.

Zbog sekvencijalne zavisnosti vremenskih koraka u RK4 metodi, podela jedne putanje po vremenu ne daje realno paralelno ubrzanje (drugi segment ne može početi bez izlaza prvog). Umesto toga, za jako i slabo skaliranje koristimo ensemble paralelizaciju: više nezavisnih trajektorija (različiti početni uslovi) koje se mogu računati potpuno paralelno. Za jako skaliranje problem fiksne veličine definišemo kao K nezavisnih trajektorija sa fiksnim brojem koraka; pri povećanju broja procesa zadržavamo isti K i merimo pad ukupnog wall-clock vremena. Za slabo skaliranje držimo konstantan posao po procesu (K trajektorija po procesu) i linearno uvećavamo ukupan broj trajektorija sa brojem procesa.

Kod jakog skaliranja imamo fiksni ukupan posao. U našem primeru većina izvršavanja programa može da se paralelizuje. Po **Amdalovom zakonu**, taj procenat smo procenili na oko **94%**. Delovi koji su strogo sekvencijalni su uglavnom

vezani za učitavanje csv fajlova i pisanje u njih, kao i za početnu inicijalizaciju. Rezultati jakog skaliranja prikazani su na slici ispod. Merenje je ponovljeno 30 puta kako bi se dobila precizna srednja vrednost i standardna devijacija. Ubrzanje je računato kao T_1/T_p gde je T_p vreme izvršavanja na p procesora a T_1 vreme izvršavanja na jednom procesoru.



Eksperiment pokazuje da vreme izvršavanja opada sa porastom broja procesa, što znači da paralelizacija uspešno raspoređuje posao između procesa.

Ubrzanje raste, ali se pri većem broju procesa primećuje zasićenje i odstupanje od idealne linije. Odstupanje od idealnog skaliranja u Pythonu je očekivano zbog:

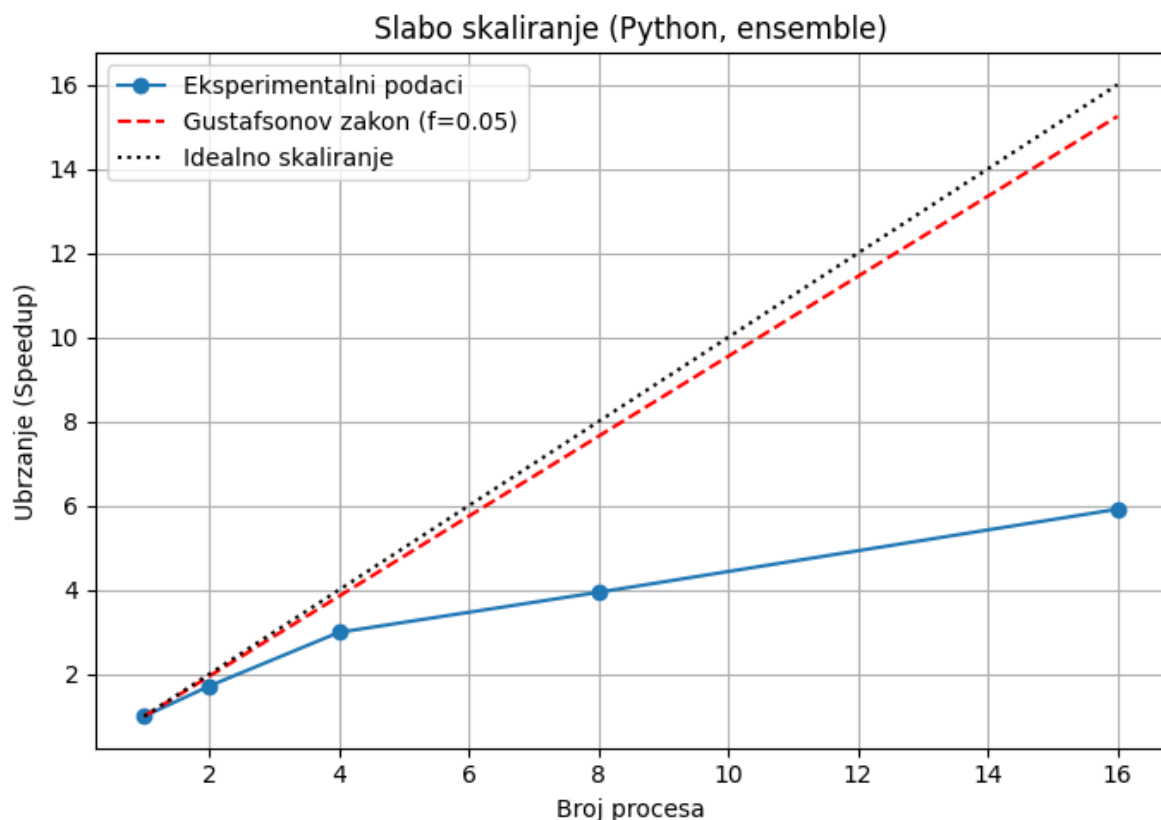
- ograničenja Global Interpreter Lock-a (GIL),
- overhead-a multiprocessing biblioteke

Broj procesora	Vreme izvršavanja	Standardna devijacija	Ubrzanje	Teorijsko ubrzanje (Amdal)
1	9.57s	0.103s	1.00	1.00
2	6.45s	0.271s	1.48	1.90
4	4.64s	0.157s	2.06	3.48
8	3.18s	0.049s	3.01	5.92
16	2.62s	0.056s	3.65	9.14

Analiza slabog skaliranja – Python

Pri slabom skaliranju ukupan posao je povećavan proporcionalno broju procesa, tako da svaki proces zadržava isti broj koraka integracije (konstantan posao po jezgru), dok ukupan posao raste približno $\sim p$ puta. Iako bi u idealnom slučaju vreme izvršavanja trebalo da ostane gotovo konstantno, dobijeni rezultati pokazuju da se ono postepeno povećava. To ukazuje na značajne režijske troškove kreiranja i komunikacije između procesa u Python-u. Ipak, oblik krive pokazuje da implementacija korektno raspoređuje posao i održava stabilno izvršavanje, dok ostvareno ubrzanje raste sporije od teorijskog Gustafsonovog zakona. Ukupno posmatrano, sistem pokazuje zadovoljavajuću skalabilnost, ali uz očigledna ograničenja zbog multiprocesnog modela i režijskih troškova. Skalirano ubrzanje se računa po formuli:

$$S_{\text{scaled}} = p * (T_1 / T_p)$$



Broj procesa	Ukupan posao (steps)	Srednje vreme izvršavanja	Standardna devijacija	Skalirano ubrzanje	Teorijsko ubrzanje (Gustafson)
1	60000	9.63	0.07	1.00	1.00
2	120000	11.22	0.78	1.71	1.95
4	240000	12.87	1.62	2.99	3.85

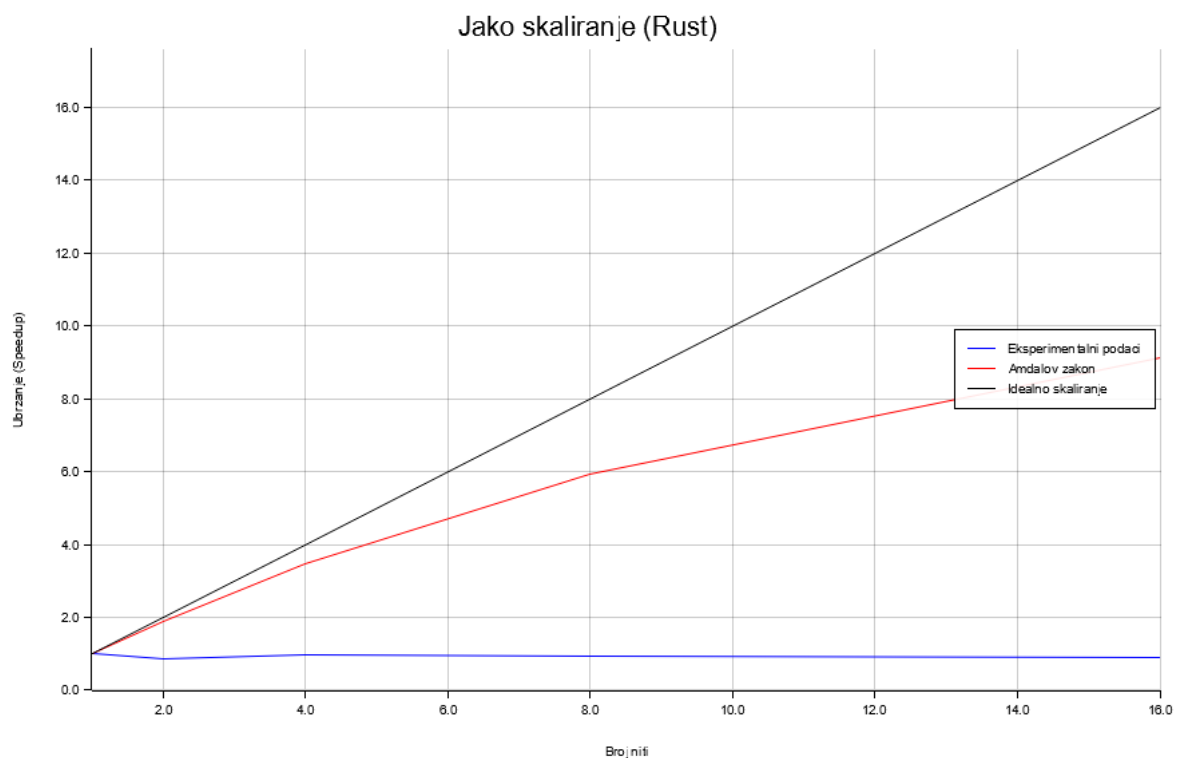
8	480000	19.53	0.36	3.94	7.65
16	960000	26.04	0.38	5.92	15.25

Analiza jakog skaliranja – Rust

Nakon analize Python implementacije, dalje razmatramo performanse rešenja napisanog u Rust-u. Rust je kompajlirani jezik poznat po visokim performansama, pa se očekuje da će ova implementacija pokazati još brže izvršavanje u odnosu na Python.

Na grafikonu je prikazano jako skaliranje Rust implementacije simulacije dvostrukog klatna. Vidimo da jako skaliranje nije postignuto.

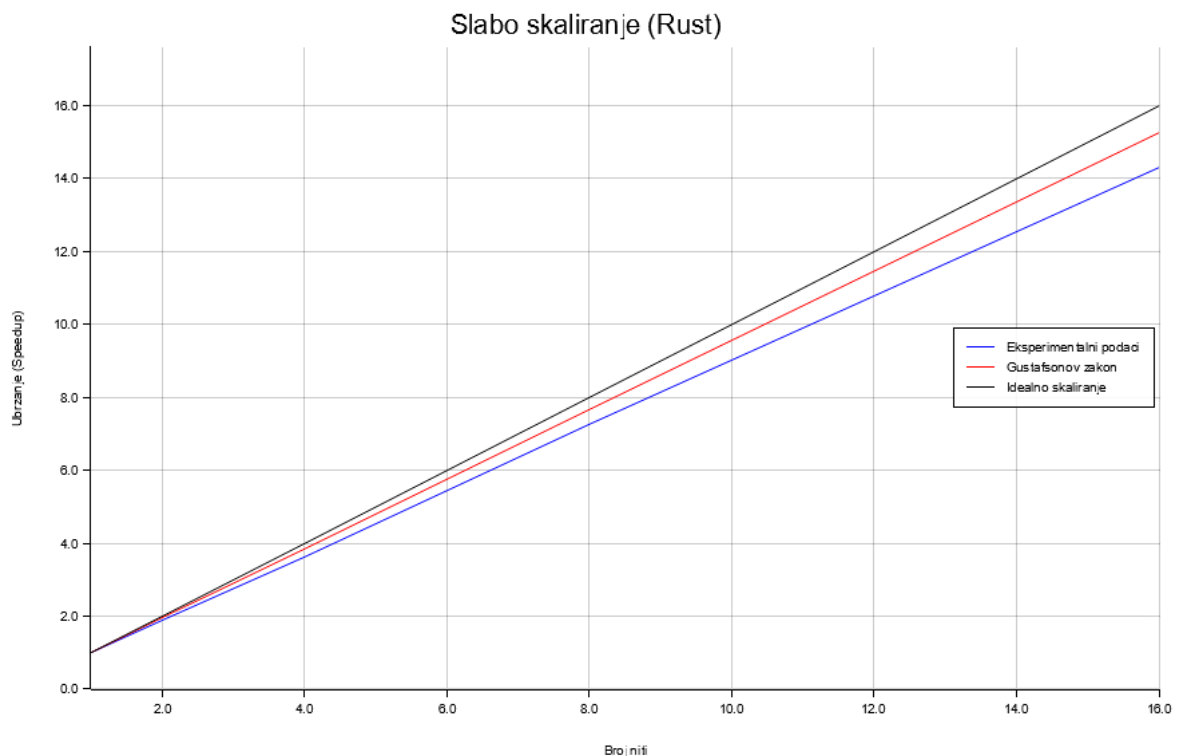
Iako Rust pruža minimalni režijski trošak i visoku efikasnost, kod ovog problema teorijsko ubrzanje se ne može ostvariti zbog prirode algoritma (zavisnost koraka). Implementacija demonstrira prednosti Rust-a u stabilnosti i kontroli performansi, ali i granice paralelizacije kod determinističkih sistema poput dvostrukog klatna.



Broj procesora	Vreme izvršavanja	Standardna devijacija	Ubrzanje	Teorijsko ubrzanje (Amdal)	Idealno ubrzanje
1	0.0099s	0.0408s	1.00	1.00	1.00
2	0.0115s	0.0810s	0.86	1.90	2.00
4	0.0102s	0.0513s	0.97	3.48	4.00

8	0.0105s	0.0631s	0.95	5.93	8.00
16	0.0111s	0.0005s	0.89	9.14	16.00

Analiza slabog skaliranja – Rust



Rezultati **slabog skaliranja**, pokazuju stabilan i gotovo linearan rast ubrzanja sa porastom broja niti. Vreme izvršavanja ostaje skoro konstantno (oko 0.01s), što ukazuje da se dodatni posao proporcionalno raspoređuje među nitima bez značajnog povećanja ukupnog trajanja. Kriva eksperimentalnih podataka nalazi se ispod Gustafsonove linije, što je očekivano usled režijskih troškova paralelizacije i ograničenja I/O podсистema.

Broj procesa	Ukupan posao (steps)	Srednje vreme izvršavanja	Skalirano ubrzanje	Teorijsko ubrzanje (Gustafson)
1	60000	0.0092	1.00	1.00
2	120000	0.0101	1.82	1.95
4	240000	0.0103	3.56	3.85
8	480000	0.0105	7.02	7.65
16	960000	0.0112	14.32	15.25

U poređenju sa Python implementacijom, Rust pokazuje **drastično kraće apsolutno vreme izvršavanja** (reda milisekundi naspram sekundi), što je očekivano s obzirom na to da se radi o kompajliranom jeziku bez GIL ograničenja.